Predictive AI for Engineers — Detailed Lesson Plan

Text & Speech Analytics · four hands-on sessions · one six-hour day (09:30–15:30) · a no-code lab

A one-day, four-session introduction to **predictive (classical) AI** through the two kinds of messy data engineers deal with every day: written **text** and spoken **speech**. This document gives the per-session timings, the demos, the case studies (with honest caveats and sources), the discussion prompts, and how the room's skeptics are handled.

The whole day deliberately covers the **predictive / analytical** half of AI — machine learning that *learns a pattern from past examples and predicts the next case*. Generative AI (ChatGPT and its relatives) is a different tool for a different job and is **out of scope here**, covered separately. Saying that out loud, early and often, is part of the plan.

Who this is for, and the one goal

The room is **experienced mechanical, aeronautical, production and quality engineers** with no programming background — people who know their machines far better than we know them. Most arrive holding one of two beliefs: **"AI can do anything"** or **"AI is marketing nonsense."** Both are wrong, and the truth is more useful than either.

The single goal: every person leaves able to **reason about an AI claim** instead of believing or dismissing it — *what did it learn from, how was it tested on unseen cases, how confident is it, and where does it get things wrong?* We earn that by showing real, **honest, imperfect** results on data that looks like theirs. One overhyped claim and the skeptics tune out for the rest of the day, so credibility is the currency we spend carefully.

Format — a no-code lab

- **No coding required.** Every code cell is pre-written. Participants only **run** cells (Shift + Enter) and read the output; in each session they change **one highlighted value** and re-run to see the result move.
- Runs in **Google Colab** in a browser — nothing to install. The notebooks are self-contained (built-in data, fixed seeds, so everyone sees the same numbers); one session also fetches a small public dataset live to prove the methods hold on real data.
- A rhythm repeated every session: **short framing → run a cell → "what just happened" → a plain-English vocabulary card → a knob to turn → recap.**
- **Mathematics is shown visually only** — a confusion-matrix grid, a similarity heatmap, a fitted line, a feature-importance bar, a spectrogram. Techniques are named for honesty (TF-IDF, logistic regression, FFT) but **no equations and no recall are expected.**

The day at a glance

#	Session (≈75 min)	The one new idea	The money-shot
1	**Text — sorting the words you already write**	A computer can't read words, only count them: turn words into numbers (TF-IDF), then it's ordinary classification	top clue-words per team — the model's reasoning in plain English
2	**Text — predict, extract & find repeat faults**	The same words-to-numbers front end answers many questions — a category, a number, a duplicate, a whole family, a part number	a similarity heatmap lighting up the same chronic fault, worded five different ways; a 2-D map of snags self-grouped into families
3	**Speech — voice → text → decision**	Speech-to-text is "audio in, text out"; then it's the text analytics from Sessions 1–2	a low word-error transcript that still gets the one part number wrong
4	**Speech — predicting from the sound itself**	A sound is already numbers; learn the pattern and a computer can *hear* a fault — no words at all	three fault types separating into clean clusters; the model leaning on the same whine and rumble a technician hears

Day schedule (six hours, 09:30–15:30, breaks included): 09:30 Session 1 · 10:45 break · 11:00 Session 2 · 12:15 lunch · 12:45 Session 3 · 14:00 break · 14:15 Session 4 (closes with the day's wrap-up & Q&A) · 15:30 finish. Each session runs ≈75 minutes; the per-session plans below are the fuller menu — the blocks marked (bonus) and some discussion are trimmed to fit. All timings flexible.

Scope — four sessions, self-contained

The workshop is **four sessions, each built around one prepared Colab notebook.** The foundations of predictive AI (the five-step shape, the core vocabulary, the train/test idea) are **introduced inside Session 1 and reused all day** — there is no separate prerequisite. The same five-step ideas apply equally to **numeric sensor data** (classification, regression, predictive maintenance), but this workshop stays focused on text and speech.

The single thread (put this on a wall poster)

Every session — text, speech, sound — is the *same five steps*. Master the shape once and the rest is variations:

dataset → split (hide some) → train → predict → measure (honestly).

Keep pointing back to it. The data changes shape (sentences, audio, a spectrum of pitches) but the picture never does. By Session 4 the room should be finishing the sentence for you.

How we win the room (the doctrine behind every session)

- **Lead with credibility, not capability.** Open by naming the two mindsets and promising the honest middle. The first example of the day is the everyday **email spam filter** — classical text AI (bag-of-words, decades old, deployed at massive scale) that nobody calls magic.
- **Always show an imperfect result and open the confusion matrix.** For experienced engineers, visible errors are a *trust* signal; a flawless demo reads as a vendor trick. Every "it can" is paired with an honest "it can't."
- **The accuracy paradox, in their language.** If real failures are 1-in-1000, a model that always says "healthy" is 99.9% accurate and useless. That is why precision/recall and the confusion matrix — not a single accuracy number — are what an engineer should demand.
- **Predict-then-reveal, one knob.** Have the room commit to a guess out loud, change one highlighted value, re-run. Active prediction plus a single controlled variable produces the "aha" that passive watching never does.
- **Show, don't state.** Charts, never equations. Engineers read Pareto and control charts fluently; one formula on screen and a no-code audience checks out.
- **Respect their expertise.** Frame every text technique as *"reading the words you already write"* and every audio technique as *"a crude version of what your ear already does."* The model just learns their existing labelling habits; it rediscovers the cues they already trust; every extraction is auditable. **Auditability is the selling point.**
- **One aerospace + one factory example per technique**, so the skill is seen as general, not an aviation trick.
- **Be explicit about real vs synthetic vs simulated** — this audience will ask. Workshop demo data is synthetic and self-contained; named datasets (ASRS, CWRU, MIMII, MAFAULDA) are real; C-MAPSS is physics-simulated.
- **Close every demo with a prompt that pulls THEIR data into the room.** "What is the label column in *your* snag logs? Who assigns it today? Where would a wrong prediction actually cost you?" Concrete prompts get debate from skeptics; "any questions?" gets silence.

Vocabulary, spread across the day

The basics are **not** front-loaded as a glossary. Each term is introduced the moment it is first needed, then reused relentlessly.

Term	First met	In one plain line
dataset, row, column	S1	a table — one row per case, columns for what we recorded
feature / label / label column	S1	features = the clues we get; label = the answer we want; the label column holds the answers
train/test split	S1	learn from most, hide some to test honestly on unseen cases
model, accuracy	S1	the learned pattern; the fraction right on the hidden set
bag-of-words, TF-IDF, stop words	S1	counting words; rare distinctive words count for more; ignore "the/and"
confusion matrix, the accuracy trap	S2	a grid of what it got right and *how* it got things wrong
classification vs regression	S2	predict a bucket vs predict a number
cosine similarity	S2	0–1 score for how much two reports use the same distinctive words
embedding · clustering (k-means)	S2	a report as a point in space; k-means groups the points into families with no labels
entity extraction (NER)	S2	pulling part numbers / components out of free text
topic modelling (LDA)	S2	grouping co-occurring words into themes with no labels
data leakage	S2	giveaway clues sneaking into the test — a too-perfect score
ASR, acoustic/language model	S3	speech-to-text; the two halves that turn sound into likely words
word error rate (WER)	S3	the speech version of accuracy — wrong + missing + extra words
keyword spotting	S3	listening for a few fixed commands instead of every word
sample rate, FFT, spectrum	S4	sound is numbers; the FFT splits it into the pitches it's made of
spectrogram, MFCC	S4	a picture of pitch over time; the "professional" sound features
supervised vs anomaly detection	S4	learn from labelled faults vs learn "normal" and flag the surprising
orders (1×, 2×)	S4	fault pitches read as multiples of shaft speed, not raw Hz

---

Session 1 — Text: sorting the words you already write

New idea: words become numbers, then it's ordinary classification.

By the end, participants can: name the five steps of a predictive-AI project; use *dataset / feature / label / train-test split / accuracy* correctly; explain (in plain words) how text becomes numbers; and read a model's top clue-words to see it is not a black box.

Time	Segment	What happens, and how it lands
0:00–0:08	**Frame the day**	Name the two mindsets. Promise the honest middle. The spam-filter anchor: you already trust classical text AI. State plainly: this is *predictive*, not generative.
0:08–0:18	**The five-step poster + first words**	Walk the spine: dataset → split → train → predict → measure. Introduce dataset/feature/label/label column as a plain spreadsheet idea.
0:18–0:23	**Into Colab**	Orient: Shift+Enter, run top-to-bottom, "Restart and run all" fixes almost everything.
0:23–0:53	**Run the prepared notebook**	Words → numbers (bag-of-words, stop words, TF-IDF) shown on two sentences first; vectorize the reports; train/test split; honest accuracy; **top clue-words per team** (oil/pressure/fluid → Hydraulic). Predict four fresh reports incl. one deliberately ambiguous — watch confidence drop.
0:53–1:00	**Turn the knob**	Change the train/test split ratio and re-run; discuss why the score wobbles on a small set.
1:00–1:12	**Where this lives at work**	Aerospace: auto-routing snags by ATA chapter; tagging safety narratives. Industrial: maintenance/IT ticket triage (an SVM hit ~85% on 3,632 HVAC maintenance requests); spam filtering.
1:12–1:22	**Discussion**	"What is the label column in *your* logs? Who assigns it today, and how long does it take?"
1:22–1:30	**Recap + bridge**	Five steps revisited; tease Session 2: sorting was the doorway — next we predict urgency, repair time, and catch repeat faults.

Skeptic handling. *"This is just if-then rules / statistics."* — Yes, and that's the point: predictive AI is disciplined pattern-finding. The top-clue-words list proves it learned something human-readable, not magic. *"Why isn't it 100%?"* — On a small set the score is a rough indicator; a model that claims 100% on real data usually peeked at the answers (proven in Session 2).

Real, citable anchors: the standard text-classification benchmark reports ~83% (Naive Bayes) and ~91% (linear model) on a real public corpus — honest mid-80s/low-90s numbers to set expectations. NASA's Aviation Safety Reporting System holds 2.3M+ de-identified narratives (free, public-domain) that look just like HAL snag/incident logs; NASA itself calls them "soft data" it does not verify — an honest data-quality lesson in one sentence.

Session 2 — Text: predict, extract & find recurring faults

New idea: the same words-to-numbers front end answers many questions — just swap the final step.

By the end, participants can: tell classification from regression and know which question each answers; read a confusion matrix for *what* a model gets wrong; explain how duplicate/recurring faults are found with no labels and how clustering groups all the snags into families (embeddings as points in space); see how part numbers are lifted out of prose and audited; and recognise data leakage (the too-perfect score).

Time	Segment	What happens, and how it lands
0:00–0:06	**Recall + preview**	One-line recap of "words → numbers." Today: predict a category, predict a number, find repeats, pull out parts.
0:06–0:12	**The snag log**	A 300-row synthetic MRO log: free-text report, system, urgency, downtime hours. Note the last is a *number*.
0:12–0:28	**Predict a CATEGORY (urgency)**	TF-IDF + logistic regression → ~81% on unseen snags. Open the **confusion matrix**: mistakes are mostly Low↔Medium; it caught 10/15 High snags and slipped High→Low only once — *rare, but it happened*, which is exactly why the High bucket keeps a human's eyes. Fold in the **accuracy trap**: a model that always says "routine" can score high yet catch zero urgent snags.
0:28–0:42	**Predict a NUMBER (downtime)**	Same TF-IDF in, swap to a regressor → off by ~1.5 h on ~6 h jobs ($R^2 \approx 0.6$). The predicted-vs-actual scatter makes "it learned the relationship" visible — and honest about the big-job misses. *This is the classification-vs-regression "aha".*
0:42–0:54	**Find RECURRING faults**	TF-IDF + cosine similarity on snags worded differently. The heatmap lights up two blocks — the same chronic fault, de-duplicated, no labels.
0:54–1:04	**Group into families (embeddings + clustering)**	Embed each snag as a point in space, then **k-means** sorts *all* of them into 5 families on a 2-D map — no labels. A few families pop out cleanly (Mechanical, Electrical, Avionics) but a catch-all swallows the rest, so agreement with the real systems is honestly **low (~0.22)** — k-means groups by language, not your org chart; you read the top words and name each. The unsupervised companion to the duplicate finder.
1:04–1:12	**Pull out PART NUMBERS**	A few readable rules extract part numbers into an auditable table, then a "most-mentioned parts" chart — free text becomes a reliability/spares signal. Every hit is checkable.
1:12–1:22	**Bonus: themes (LDA) + the honest-test trap**	One pre-baked topic view (some topics map to systems, some to severity language — *you* name them). Then the same pipeline on a real public corpus: ~84% honestly vs ~95% when giveaway headers leak in. *How was it tested, and could it have peeked?*
1:22–1:30	**Discussion + recap**	"Where would a wrong prediction be expensive vs cheap for you? Do your historical logs exist *labelled*?"

Skeptic handling. *"Can it replace our engineers?"* — No: it hands you a prioritised list and a reason; a human decides. The confusion matrix and the High→Low slip make the limits measurable, which is exactly why it's trustworthy.

Real, citable anchors (with honest caveats):

- **Chronic/recurring-defect detection** is a live commercial product in aviation MRO. Its headline figures — used by >25% of the world fleet, ~40% of defects unflagged due to inconsistent coding, 90%+ code prediction — are **vendor-claimed, not independently audited**. Lead with the capability (de-duplicating differently-worded faults), not the percentages.
- **Part-number extraction:** a US defence study lifted part-number extraction accuracy (F1) from ~3% (plain pattern-matching) to ~95% (a trained extraction model) — an honest *mixed* result (rigid codes were easy for both).
- **Defence precedent:** a national defence organisation mined two years of military-aircraft maintenance and pilot free-text to surface "lead indicators" of defects — the closest published analogue to HAL's situation.
- **Topic modelling, honestly:** a NASA-data study recovered >70% of hand-curated reports on an emerging theme via topic modelling — *but* a separate study concluded off-the-shelf topic tools were "insufficient for sense-making" on their own. A useful map, not a verdict.
- **Industrial transfer:** national consumer-complaint databases (~1.9M free-text records, public) are the warranty-mining analogue.

Indian context to mention lightly: after a 2025 accident, India's DGCA is reported to be tightening repetitive-defect reporting (a defect recurring three times within 15 flight cycles) — **press-reported/draft**, so cite it as motivation, not settled regulation.

Session 3 — Speech: voice → text → decision

New idea: speech-to-text is "audio in, text out"; then it's the text analytics you already know.

By the end, participants can: describe ASR as the same predict-from-examples idea with sound as the input; read WER as the speech version of accuracy and explain why a low WER can still be unsafe; and say why a small fixed vocabulary (keyword spotting) is more reliable than full transcription.

Time	Segment	What happens, and how it lands
0:00–0:08	**Frame**	The last messy data type: spoken words. The pipeline: speak → transcribe → decide. Draw the classical→modern line (HMM-GMM → deep models, ~2012) explicitly so nobody thinks "this is ChatGPT" — both are *predictive*, not generative.
0:08–0:30	**Run the speech pipeline**	Manufacture a spoken report (text-to-speech, no mic needed) → transcribe it → route it with the Session-1 classifier, with a confidence bar. *(First cell installs two tools, ~1 min — the only setup of the day.)*
0:30–0:45	**Measure it honestly (WER)**	A self-contained WER demo: a noisy transcript scores a *worse* 15% yet stays harmless; a "cleaner" 8% transcript quietly turns one part number into another — the one word that matters, now wrong. **Low error rate ≠ safe.** Rule: a human verifies numbers and serials.
0:45–0:55	**Keyword spotting**	Listening for a 4-command vocabulary inside messy transcripts — robust to stutters and filler, and it leaves the no-command line alone. Why hands-free shop-floor tools use wake-words, not dictation.
0:55–1:08	**Where it works, where it breaks**	Hard by default (accents, jargon, radio noise), strong with domain data: a European air-traffic-control speech project reached >95% (controllers) / >90% (pilots) word recognition *after* domain training, ~97% on callsigns when fused with radar. Open ATC corpora exist (clean and noisy) for exactly this.
1:08–1:20	**Discussion**	"Where do people *speak* information that never gets written down — handovers, inspections, calls? What's the one word in your world that must never be mis-heard?"
1:20–1:30	**Recap + bridge**	Speech → text → decision, honestly measured. Tease Session 4: next we throw the words away and predict from the *sound itself*.

Skeptic handling. *"ASR is hype — it never gets my accent."* — Agree out loud, then show the WER demo and the domain-adaptation counter-evidence: out of the box it struggles; trained on your domain it gets strong. Honesty about the limit is what makes the capability believable.

Session 4 — Speech: predicting from the sound itself

New idea: a sound is already numbers; learn the pattern and a computer can *hear* a fault — no words.

By the end, participants can: explain that a sound is just numbers and that a few features (energy, low/high bands, dominant pitch) capture what the ear hears; tell supervised classification from unsupervised anomaly detection and which fits the realistic "faults are rare" case; and read fault pitches as orders of shaft speed.

Time	Segment	What happens, and how it lands
0:00–0:08	**The technician's ear**	A seasoned tech walks past and says "that bearing's going" — from the sound. Can a computer learn that? Same idea, last time: numbers → pattern → judge.
0:08–0:22	**Sound is numbers; listen, then featurise**	Sample rate → a clip is a list of numbers. *Play* healthy / bearing / imbalance clips. The FFT splits a sound into its pitches (a prism); five features capture loudness, low rumble, high whine, dominant pitch, brightness.
0:22–0:34	**See the pattern, then classify**	Scatter of low-band vs high-band energy → three clean clusters. Train/test split → ~88% on unseen clips. **Feature importance** shows the model leaning on the whine and the rumble — the very cues the ear uses.
0:34–0:44	**Diagnose a brand-new machine**	Play a fresh clip, get a fault call + confidence. Turn the knob: try other faults / seeds; widen the bearing range and watch accuracy soften — honest.
0:44–0:58	**When you have no fault examples**	The realistic aerospace case: lots of healthy sound, few faults. Train an anomaly detector on **healthy only** (AUC ≈ 0.94); the histogram shows healthy left, faults right of the alarm line. Move the line: ~5% false alarms catches ~96% of imbalance and ~⅔ of bearing faults — *no free lunch*, and the trade-off is visible.
0:58–1:10	**Real datasets + orders**	Name the real, benchmarked field: CWRU bearings, MAFAULDA (with a mic channel), MIMII / the DCASE challenge (honest AUCs ~0.54–0.96). Read pitches as **orders** (imbalance 1×, misalignment 2×), not raw Hz.
1:10–1:20	**Where it lives in aerospace**	Helicopter HUMS detects ~70% of the failure modes it monitors (strong, explicitly imperfect); engine health monitoring (one OEM monitors 8,000+ aircraft); acoustic-emission NDT for cracks — same recipe: signal → features → classify/flag.
1:20–1:30	**The day in one picture + close**	Numbers, text, speech, sound — it was always the same five steps. The "neither magic nor nonsense" close: you can now *question* an AI claim. Hand off to the generative-AI sessions to come.

Skeptic handling. *"But we don't have failure labels."* — Exactly why the anomaly-detection demo matters: learn normal, flag the surprising, no fault examples needed. *"Why not 100%?"* — public benchmark scores run from ~54% to ~96%; a flawless demo is the thing to distrust.

Case studies (and how to cite them honestly)

Case	What it really is	How to say it
Airbus Skywise — A330neo bleed valve (EASA emergency AD, Aug 2022)	Cross-fleet data analysis flagged a software bug over-stressing a valve pin; caught *before* any accident	The best honest safety story for the room — predictive analytics as an engineering early-warning, not cost-savings hype. (Skywise now connects 12,000+ aircraft — quote the live figure.)
Delta TechOps predictive maintenance	Maintenance-caused cancellations fell from 5,600+ (2010) to ~55 (2018), >95% success on pending-failure predictions	Powerful — but **self-reported**, not independently audited. Model the "how was this measured?" discipline.
Aviation chronic-defect NLP	Reads maintenance logbook free-text to recognise the same chronic defect described differently	**Vendor-claimed** figures; lead with the capability, flag the numbers as unaudited.
Air-traffic-control speech recognition	Domain-adapted ASR for ATC	>95% / >90% word recognition *after* domain training; ~97% callsigns with radar context. Frame as "hard by default, strong with domain data."
Machine-sound anomaly benchmark (DCASE / MIMII)	Public benchmark for detecting machine faults from sound using only normal data	The anti-hype anchor: honest AUCs ~0.54–0.96 across machine types.
Engine health monitoring	Streaming engine parameters to forecast component replacement, underpinning "power-by-the-hour" service	One OEM reports monitoring 8,000+ aircraft. Shows the business model that forces honest models.
Indigenous fighter-fleet predictive maintenance (India, 2026)	A prognostic maintenance programme for a ~270-strong fleet, with AI and real-time sensor monitoring	"This is happening here" context — but it is **sensor/prognostics**, not text/audio; cite as context only.

On HAL itself: there is **no verified public evidence** of HAL running predictive ML in production. Frame everything as **"applicable to HAL's operations,"** not "HAL already does this." HAL's strongest latent asset is its own historical snag/overhaul logs — *if* they are clean and labelled.

Public datasets (Colab-usable or citable)

Dataset	What	Use	Source
NASA ASRS	2.3M+ de-identified aviation safety narratives (free text + synopsis)	S1–S2 text routing / topics	asrs.arc.nasa.gov (public domain)
MaintNet	6,169 real aviation maintenance logbook entries + tools	S1–S2 "cleaning beats the model"	aclanthology.org/2020.coling-demos.2
FAA SDRS	~1.7M service-difficulty records, free-text + structured	S1–S2 trend/routing	faa.gov/av-info/download_SDR
20 Newsgroups	~18k real documents, one-line fetch	S2 real-data proof + the header-trap	built into scikit-learn
ATCOSIM / ATCO2	Clean / noisy air-traffic-control speech corpora	S3 realism	spsc.tugraz.at · arxiv.org/abs/2211.04054
Speech Commands	1-sec spoken-word clips	S3 keyword spotting	arxiv.org/abs/1804.03209
CWRU Bearing	The bearing-fault benchmark	S4 "real, benchmarked field"	engineering.case.edu/bearingdatacenter
MIMII	Real valve/pump/fan/slide-rail sound + factory noise	S4 anomaly detection	zenodo.org/records/3384388
MAFAULDA	1,951 runs: imbalance/misalignment/bearing, incl. a mic channel	S4 maps to hand-diagnosed faults	www02.smt.ufrj.br/~offshore/mfs
NASA C-MAPSS	*Simulated* turbofan run-to-failure (remaining useful life)	S4 regression bridge (label it simulated)	data.nasa.gov

Heavy downloads (CWRU, MIMII, MAFAULDA) stay out of the live lab — they are named and linked for credibility; the synthetic demos are the floor, not the ceiling.

Foundations & optional material

The five-step foundations — *dataset, feature, label, label column, train/test split, accuracy, the confusion matrix* — are introduced inside **Session 1** and reused all day; there is no separate prerequisite. The same ideas apply just as well to **numeric sensor data** (classification, regression, predictive maintenance with feature importance and anomaly detection), but the workshop stays focused on text and speech.

Lab logistics & pre-flight

- A computer lab with **internet**, one machine per participant (pairs are fine), able to reach Google Colab.
- A **Google account** per participant (personal works), or the upload fallback.
- A projector for the framing slides and the money-shot charts.
- **Pre-flight (day before):** run **Session 3** end-to-end once — its first cell installs the speech tools (~1 minute) and downloads a small model on first transcription; confirm the network allows it. Everything else needs no install.
- **The one reset that fixes almost everything:** *Runtime → Restart and run all.* Say it often.

Sources & further reading (selected, verified)

Topic	Source
Spam filtering as classical text AI	en.wikipedia.org/wiki/Naive_Bayes_spam_filtering
Honest text-classifier accuracy + the header trap	scikit-learn.org (20 Newsgroups tutorial & dataset notes)
Work-order text classification (SVM ~85% / 3,632 HVAC)	ASCE J. Arch. Eng. (2022), doi 10.1061/(ASCE)AE.1943-5568.0000522
Aviation Safety Reporting System & "soft data" caveat	asrs.arc.nasa.gov
Part-number extraction (3%→95%)	apps.dtic.mil/sti/trecms/pdf/AD1157022.pdf
Defence maintenance-text NLP (ICAS 2024)	icas.org/icas_archive/icas2024 (#0083)
Topic modelling on safety narratives (>70% recovery; "insufficient for sense-making")	ntrs.nasa.gov/citations/20210011508 ; /20205003750
Word Error Rate definition	en.wikipedia.org/wiki/Word_error_rate
Air-traffic-control speech recognition (HAAWAII)	cordis.europa.eu/article/id/442201
Deep models replace GMM (~2012)	cs.toronto.edu/~hinton/absps/DNN-2012-proof.pdf
Machine-sound anomaly benchmark (DCASE 2020, AUC ~0.54–0.96)	ar5iv.labs.arxiv.org/abs/2006.05822
Helicopter HUMS (~70% of monitored failure modes)	skybrary.aero/articles/vibration-health-monitoring-vhm
Skywise A330neo early warning (EASA AD 2022)	theaircurrent.com (Skywise A330neo)
Engine health monitoring (power-by-the-hour)	rolls-royce.com (civil aerospace / EHM)

All quantitative claims above were fact-checked; figures that are vendor-claimed, self-reported, simulated, or press-reported/draft are flagged as such in the text — modelling the exact "where did this number come from?" discipline the workshop teaches.